A Case Study of Staged Metric-Gated GRPO for Visual Numeric Reasoning

Kunal Ghosh
Purdue University
West Lafayette, Indiana, USA
ghosh178@purdue.edu
kg.aero@gmail.com

Abstract

We present a code-grounded case study of a staged metric-gated GRPO pipeline for visual numeric reasoning. The repository wraps `Qwen2.5-VL-7B-Instruct` with four main design choices: phase-wise curriculum over filtered MathVista subsets, a strict `<REASONING>` / `<SOLUTION>` response contract, a checkpoint-metric controller that preserves or raises structure-oriented reward weights when formatting or completion degrades, and alias-based checkpoint selection because later RL checkpoints are not always better. The report is intentionally narrow. Rather than claiming a new GRPO objective or a benchmark result, we analyze the archived artifacts that the repository itself produced during training and checkpoint selection. Under that internal evaluation protocol, the archived best large Phase C checkpoint and the archived best dedicated Phase D checkpoint each record exact match and tolerance accuracy of 0.75, up from 0.375 for a saved pre-refactor baseline snapshot, while maintaining parseability 1.0 and zero truncation on the monitored subset. The most important pattern in those artifacts is structure first, correctness later: smoke runs stabilize parseability and output form, and only then do larger continuation runs reach the 0.75 milestone. The contribution of the paper is therefore a reproducible case study of staged RL engineering for multimodal numeric QA, not a claim of broad empirical superiority.

Code: https://github.com/kgaero/RL_GSPO_Qwen2.5VLM

1 Introduction

Reinforcement-learning fine-tuning for language and vision-language models is often summarized as "add reward, get better accuracy." In small multimodal projects, the reality is usually messier. Before correctness reward becomes informative, the policy must already emit outputs that are parseable, non-truncated, and structurally compatible with the evaluator. If those preconditions are unstable, RL updates can become difficult to interpret because an apparent correctness failure may in fact be a formatting or completion failure.

This paper studies one concrete repository that tackles that problem with staged metric-gated GRPO. The repository does not modify the GRPO loss itself. Instead, it builds a training stack around GRPO: filtered curriculum stages, a strict response contract, rule-based reward adaptation keyed to checkpoint metrics, and checkpoint aliases that allow continuation from the best structural or composite checkpoint rather than always from the most recent one. Our goal is to document what that stack does, what the saved artifacts say about it, and what kinds of claims the evidence can honestly support.

That framing makes this paper narrower than a conventional empirical method paper. We are not presenting a broad benchmark study, a new theorem, or a many-baseline comparison. We are presenting a repository case study. The value of that case study is that the code is unusually auditable for a small RL project: controller states are saved, checkpoint aliases are explicit, reward weights are persisted, and the main milestones are recoverable from repository artifacts. This makes the project a useful example of how staged RL can be organized when compute is limited and structural failures dominate early training.

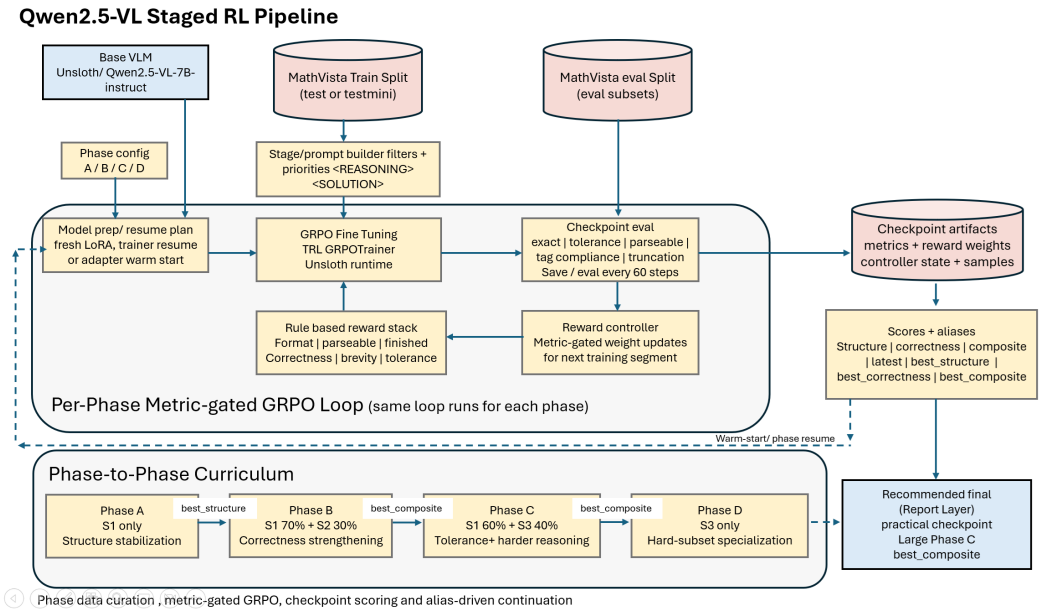

Figure 1: Overview of the repository's staged metric-gated GRPO loop. The pipeline couples phase-specific data curation, GRPO fine-tuning, checkpoint-side evaluation, reward-weight adaptation, and alias-based continuation. The contribution analyzed here is the training stack around GRPO rather than a new GRPO loss.

Figure 1 summarizes the loop. A base VLM is prepared with LoRA adapters, trained on a stage-filtered phase mixture, evaluated checkpoint by checkpoint, and then fed back into a controller that adjusts the next segment's reward weights. Table 1 shows that the curriculum is implemented as overlapping filtered views over the same MathVista split rather than as disjoint datasets.

Case-study contributions.

1. We document a staged GRPO training stack for visual numeric reasoning that explicitly separates structure stabilization from later correctness improvement.
2. We formalize the repository's metric-gated reward controller and checkpoint aliasing logic so they can be read as an auditable algorithm rather than as scattered implementation details.
3. We summarize the archived training trajectory that the repository itself used for continuation decisions, including the 0.75 exact-match milestone reached by the archived best large Phase C and dedicated Phase D checkpoints.
4. We clarify the scope of the evidence: these numbers are internal checkpoint-side measurements suitable for a reproducible case study, not a leaderboard-style claim.

2 Related Work

RL fine-tuning and preference optimization. Recent RLHF and preference-optimization work has primarily focused on new objectives or more efficient learning regimes. DeepSeekMath introduced GRPO as a group-relative alternative to PPO for mathematical reasoning (Shao et al., 2024). At ICLR 2025, SPPO proposed a self-play preference objective with a stronger theoretical framing than pairwise preference losses (Wu et al., 2025), while Asynchronous RLHF studied speed-quality trade-offs in off-policy online RLHF (Noukhovitch et al., 2025). Our paper is orthogonal to those contributions. We do not alter the underlying GRPO objective; we study how one repository wraps GRPO with curriculum, metric-gated reward scheduling, and checkpoint-selection logic in a failure-prone multimodal setting.

Multimodal mathematical reasoning. MathVista exposed the difficulty of mathematical reasoning in visual contexts and remains a useful benchmark for diagram, chart, table, and numeric QA behavior (Lu et al., 2024). More recent ICLR papers have attacked that problem with larger synthetic corpora, progressive curricula, or more explicit visual reasoning mechanisms (Zhang et al., 2025; Qi et al., 2025). Our scope is narrower. We do not introduce a new dataset or a new multimodal reasoning architecture. Instead, we analyze how a standard pretrained VLM behaves under a staged RL wrapper that prioritizes structural reliability before harder correctness pressure.

Curriculum and adaptive reward control. The repository is closest in spirit to curriculum and adaptive-control approaches that treat training as a sequence of regimes rather than as one stationary objective. The distinctive feature here is the granularity of adaptation: checkpoint metrics trigger reward changes directly. That makes the controller easy to audit, which is especially useful in a case-study setting where interpretability of the training loop matters as much as the raw score.

3 Problem Setup and Design Goals

We study a policy $\pi_\theta(c \mid x)$ that maps a visual question-answering input x to a completion c. The repository does not treat success as a single scalar notion of accuracy. Instead, it decomposes success into a sequence of preconditions:

1. the model must emit the required tagged structure,
2. the final answer must be parseable,
3. the answer must finish within the completion budget, and
4. the parsed answer must be correct under exact or tolerance-based matching.

The practical question is therefore not only how to optimize correctness, but when correctness reward becomes worth emphasizing. If structure is unstable, correctness reward is sparse and often misleading because malformed or truncated outputs never reach a meaningful matching stage.

The implementation makes three concrete design choices.

- **Easy-to-hard data staging.** The policy first sees easier integer-valued numeric problems before harder float and geometry-heavy subsets are emphasized.
- **Metric-gated rewards.** Reward weights change only after checkpoint evaluation, using measured failure modes rather than a fixed epoch schedule.
- **Alias-based continuation.** Checkpoint continuation is based on structure, correctness, or composite aliases rather than assuming the latest checkpoint is best.

These choices are modest in isolation, but together they define the engineering story of the repository.

4 Method

4.1 Task, response contract, and phase structure

The code targets MathVista-style visual question answering in which the answer is primarily numeric and free-form. Each prompt is wrapped with a strict output contract:

```
<REASONING> ...  </REASONING>
<SOLUTION> single numeric answer </SOLUTION>
```

This contract matters because both the reward functions and the evaluator depend on extracting a unique solution string from the completion. The pipeline also contains a separate multi-choice scaffold, but that branch is disabled by default and is not part of the main case-study evidence.

The curriculum is organized into stages and phases. Stages are filtered subsets over the same dataset. In the default configuration:

Filter	S1 Easy Numeric	S2 Medium / Float	S3 Hard Numeric
Stage ID	stage1_easy_numeric	stage2_float_numeric	stage3_hard_numeric
Goal	easy numeric stabilization	moderate precision and broader numeric coverage	harder multistep numeric reasoning
Question type	free_form	free_form	free_form
Answer type	integer	integer, float	integer, float
Language	english	english	english
Answer mode	numeric_free_form	numeric_free_form	numeric_free_form
Context families	synthetic scene table natural image	table chart plot scientific figure natural image	geometry diagram plot scientific figure abstract scene
Skills (any)	arithmetic reasoning statistical reasoning numeric commonsense	arithmetic reasoning statistical reasoning scientific reasoning numeric commonsense	geometry reasoning algebraic reasoning scientific reasoning arithmetic reasoning statistical reasoning
Hard only	False	False	True
Priority bias	ctx: synthetic scene ctx: table skills: arithmetic reasoning skills: statistical reasoning skills: numeric commonsense grades: elementary school grades: daily life	ctx: table ctx: chart ctx: plot ctx: scientific figure skills: statistical reasoning skills: scientific reasoning skills: arithmetic reasoning	ctx: geometry diagram ctx: scientific figure ctx: plot skills: geometry reasoning skills: algebraic reasoning skills: scientific reasoning grades: high school grades: college

Table 1: Code-derived filter definitions for the three numeric curriculum stages used in the main training path. The stages overlap and are implemented as filtered views over the same MathVista split, not as fixed disjoint datasets.

- **Stage 1** restricts to easier English free-form integer questions on synthetic scenes, tables, and natural images.
- **Stage 2** broadens to integer and float answers, adding tables, charts, plots, and scientific figures.
- **Stage 3** emphasizes harder geometry, scientific-figure, plot, and abstract-scene problems.

Phases then mix these stages with different reward emphases:

- **Phase A**: Stage 1 only, structure stabilization.
- **Phase B**: 0.7 Stage 1 + 0.3 Stage 2, early correctness strengthening.
- **Phase C**: 0.6 Stage 2 + 0.4 Stage 3, harder reasoning and tolerance reward.
- **Phase D**: Stage 3 only, hard-subset specialization with a longer completion budget.

4.2 Reward decomposition

For a completion c, the repository computes six per-sample reward components:

$$R(c) = \sum_{j=1}^{6} w_j r_j(c), \tag{1}$$

where the components correspond to `format`, `parseable`, `finished`, `correctness`, `brevity`, and `tolerance`. The initial weights are phase-specific. Phase A places relatively high weight on structure and completion; Phase C raises correctness and activates tolerance reward.

The implementation never allows structure rewards to decay fully to zero. That is the core design intuition of the repository: even when exact match becomes the main target, structure remains a necessary condition for receiving useful reward. The exact code-aligned reward definitions are reproduced in the supplement.

4.3 METRIC-GATED REWARD CONTROL

The reward controller updates weights after each checkpoint evaluation using explicit rules derived from observed metrics. Let p denote parseable-answer rate, s solution-tag compliance, r reasoning-tag compliance, m malformed rate, t truncation rate, and e exact match. The repository applies the following guards:

$$p < 0.85 \Rightarrow w_{\text{parseable}} \leftarrow \max(w_{\text{parseable}}, 0.75), \tag{2}$$

$$(s < 0.90) \ \vee \ (r < 0.88) \ \vee \ (m > 0.10) \Rightarrow w_{\text{format}} \leftarrow \max(w_{\text{format}}, 0.75), \tag{3}$$

$$(t > 0.15) \ \vee \ (\bar{\ell} > 0.85L) \Rightarrow w_{\text{finished}} \leftarrow w_{\text{finished}} + 0.25, \tag{4}$$

where $\bar{\ell}$ is average completion length and L is the completion budget.

Correctness is escalated only when structure is already stable. Stability requires

$$p \geq 0.92, \quad s \geq 0.95, \quad r \geq 0.93, \quad m \leq 0.05, \quad t \leq 0.08, \tag{5}$$

for a two-checkpoint window. If that condition holds and exact-match improvement over the window is below 0.02, the controller increases correctness weight by 0.50. All weights are then clamped to configured bounds. The rule set is intentionally simple enough to audit directly in saved checkpoint artifacts.

4.4 CHECKPOINT-SIDE EVALUATION AND ALIAS-BASED SELECTION

Instead of treating the most recent checkpoint as the default continuation point, the repository evaluates every saved checkpoint and computes three aggregate scores:

$$S_{\text{struct}} = 0.35\,p + 0.20\,s + 0.20\,r - 0.15\,m - 0.10\,t, \tag{6}$$

$$S_{\text{corr}} = 0.70\,e + 0.20\,a_{\text{tol}} + 0.10\,p, \tag{7}$$

$$S_{\text{comp}} = 0.45\,e + 0.15\,a_{\text{tol}} + 0.15\,p + 0.10\,s + 0.05\,r - 0.05\,m - 0.05\,t, \tag{8}$$

where a_{tol} is tolerance accuracy. These scores back four aliases: `latest`, `best_structure`, `best_correctness`, and `best_composite`. Cross-phase continuation typically resumes from `best_structure` or `best_composite`, while same-phase restarts may use full trainer resume from `latest`. This distinction matters in the repository because the code supports both full trainer-state restoration and LoRA-only warm start.

5 REPOSITORY SETTING AND ARCHIVED EVALUATION PROTOCOL

Dataset and model. The implementation targets the Hugging Face release of MathVista[1] and loads `Qwen2.5-VL-7B-Instruct`[2] as the base VLM. For context we cite the MathVista benchmark paper (Lu et al., 2024) and the Qwen2.5-VL technical report (Bai et al., 2025).

Runtime profile. The repository's main successful runs use the conservative `kaggle_t4` profile: 4-bit loading, LoRA rank 8, max sequence length 1280, image size 336, gradient accumulation 4, two sampled generations per prompt during RL, max prompt length 320, and max completion length 64 (or 96 in Phase D). The trainer uses TRL's `GRPOTrainer` with sequence-level importance sampling and `dr_grpo` loss.

Archived evaluation protocol and reporting choice. All quantitative values reported in this paper are taken from archived checkpoint-side artifacts generated during the original runs. Those archived values are the ones that the repository itself used for controller decisions and checkpoint aliasing. In smoke runs the code trains on `testmini` and also evaluates on `testmini`; in the larger continuation the code trains on `test` and evaluates on `testmini`. Under the Kaggle profile, evaluation uses one sampled completion per prompt and caps each subset at two examples. We therefore treat the numbers below as internal measurements for a reproducible repository case study, not as benchmark-comparable MathVista results.

[1] https://huggingface.co/datasets/AI4Math/MathVista
[2] https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct

Parameter	Default	New Value	Function Name	Phase
max_seq_length	16384	1280	FastVisionModel.from_pretrained	all
image_size	512	336	build_stage_dataset	all
gpu_memory_utilization	0.8	0.65	FastVisionModel.from_pretrained	all
LoRA config (lora_rank / max_lora_rank / lora_alpha)	16 / None / 16	8 / 8 / 8	FastVisionModel.get_peft_model	all
compilation_config	unset	level 3 PIECEWISE	FastVisionModel.from_pretrained	all
Training budget (gradient_accumulation_steps / num_generations)	2 / 4	4 / 2	GRPOConfig	all
Prompt budget (max_prompt_length / max_completion_length)	1024 / 256	320 / 64	GRPOConfig	A–C, E
Eval sampling (num_samples_per_prompt / max_eval_examples_per_subset)	4 / None	1 / 2	SamplingParams	all
Phase D completion budget (max_completion_length)	320	96	GRPOConfig / SamplingParams	D only

Table 2: Code-derived implementation summary of the low-memory profile used by the archived successful runs. The **Function Name** column lists the concrete runtime method or config class that consumes the setting, rather than the config-builder that defines its default. The smoke experiments and the larger continuation therefore share the same Kaggle T4-safe operating regime.

Table 3: Archived milestone checkpoints under the repository's internal evaluation protocol. These are the saved checkpoint-side metrics used by the original continuation logic.

Checkpoint	EM	Tol	Parse	Malf.	Trunc.	Comp.
Pre-refactor baseline	0.375	0.375	0.641	0.359	0.242	0.405
Smoke Phase A best	0.500	0.500	0.500	0.500	0.500	0.400
Smoke Phase B best	0.500	0.500	1.000	0.000	0.000	0.600
Smoke Phase C best	0.500	0.500	1.000	0.000	0.000	0.600
Large Phase C best	**0.750**	**0.750**	**1.000**	**0.000**	**0.000**	**0.750**
Dedicated Phase D best	**0.750**	**0.750**	**1.000**	**0.000**	**0.000**	**0.750**

Reproducibility. The repository saves checkpoint-side artifacts including metrics, subset metrics, reward weights, controller state, controller decision logs, prompt-level records, and sample-level JSONL traces. It also includes 26 unit tests covering configuration, data filtering, parsing, checkpointing, evaluation aggregation, controller behavior, and runtime compatibility.

6 Archived Results

6.1 Structure improves before correctness in the archived trajectory

Table 3 shows the clearest pattern in the saved artifacts. The archived pre-refactor baseline snapshot reaches only 0.641 parseability with 0.359 malformed rate and 0.242 truncation. Smoke Phase A shortens outputs, but structure remains poor. The decisive structural change appears in archived Smoke Phase B: parseability, reasoning-tag compliance, and solution-tag compliance all reach 1.0, while malformed and truncation drop to zero. Exact match, however, remains at 0.5. Within the repository's own training-time measurements, that is the crucial transition: structural reliability saturates before the main correctness gain arrives.

Figure 2 makes the transition visually explicit. Once structural metrics saturate, the archived trajectory stops looking like "make outputs valid" and starts looking like "make valid outputs correct." That reading is consistent with the controller design: early checkpoints spend reward mass on formatting,

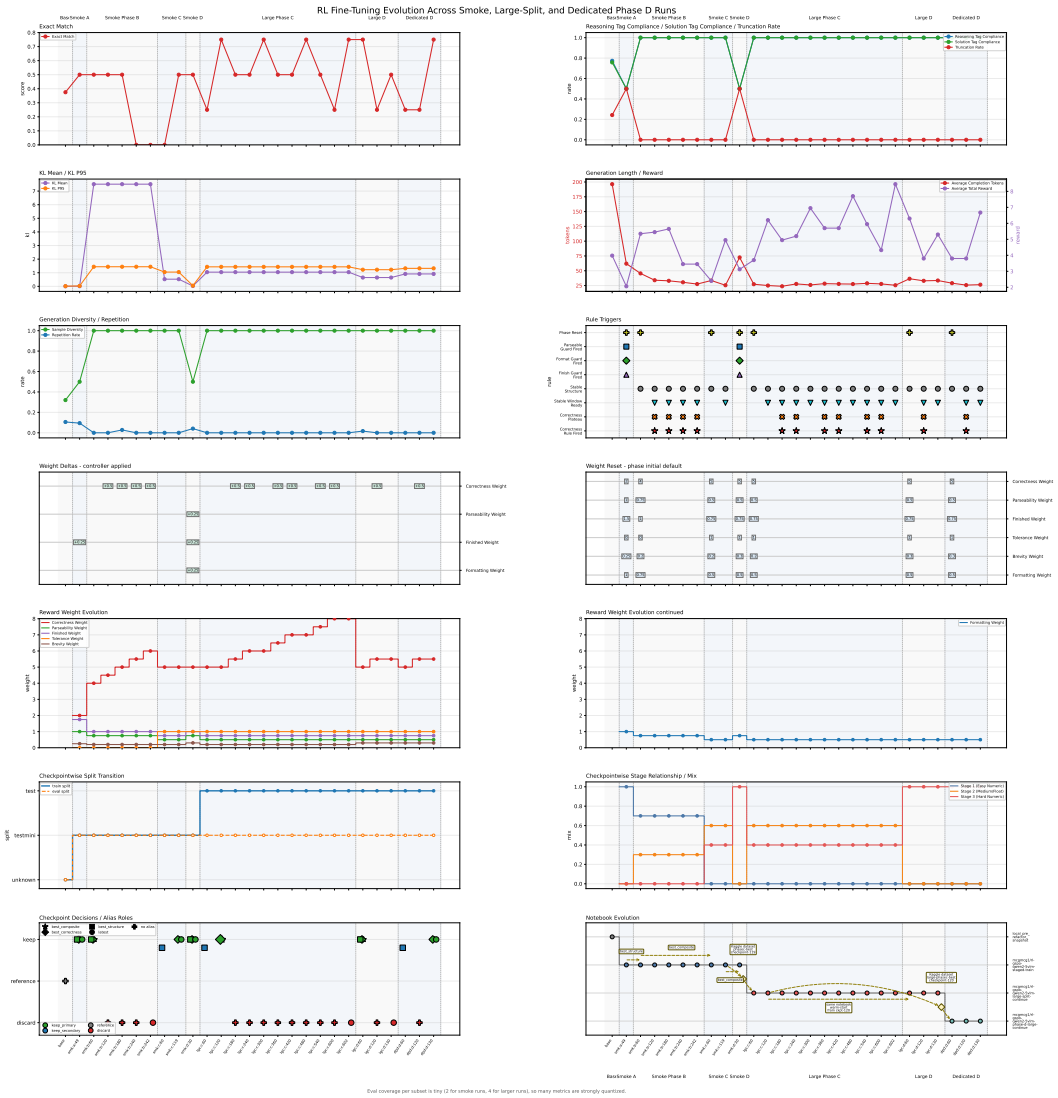

Figure 2: Training evolution across archived milestone checkpoints. The main qualitative pattern is structure first, correctness later: structural metrics stabilize by the smoke Phase B / Phase C regime, while the archived 0.75 exact-match milestone appears only after larger continuation.

Table 4: Comparison between the saved pre-refactor baseline snapshot and the archived best large Phase C checkpoint. In this paper the table should be read as an outcome summary for the archived run artifacts, not as a benchmark table.

Core Accuracy			
Eval	**Base**	**Trained**	**% Improvement**
Exact Match	**37.5%**	**75.0%**	**+100.0%**
Tolerance Accuracy	37.5%	75.0%	**+100.0%**
Structure / Completion			
Eval	**Base**	**Trained**	**% Improvement**
Parseable Rate	64.1%	100.0%	**+56.1%**
Reasoning Tag Compliance	**77.3%**	**100.0%**	**+29.3%**
Solution Tag Compliance	**75.8%**	**100.0%**	**+32.0%**
Well-Formed Rate	64.1%	100.0%	**+56.1%**
Malformed Rate (lower better)	35.9%	0.0%	**+100.0%**
Completion Success Rate	75.8%	100.0%	**+32.0%**
Truncation Rate (lower better)	24.2%	0.0%	**+100.0%**
Generation			
Eval	**Base**	**Trained**	**% Improvement**
Average Completion Tokens (lower better)	196.3	25.0	**+87.3%**
Repetition Rate (lower better)	10.6%	0.0%	**+100.0%**
Sample Diversity	32.0%	100.0%	**+212.2%**
Scores / Reward			
Eval	**Base**	**Trained**	**% Improvement**
Average Total Reward	3.98	6.2	**+55.7%**
Structure Score	45.2%	75.0%	**+65.8%**
Correctness Score	40.2%	77.5%	**+93.0%**
Composite Score	40.5%	75.0%	**+85.0%**

Source: Pre-Refactor Baseline (`base`) vs Large Phase C checkpoint-120 (`lgc:c:120`) from the repository's archived checkpoint tables.

parseability, and clean completion; later checkpoints inherit those gains and then receive more correctness pressure.

6.2 The archived 0.75 milestone appears in larger continuation

The strongest archived checkpoints in the repository are the large-split Phase C model at checkpoint 120 and the dedicated Phase D model at checkpoint 130. Relative to the saved baseline snapshot, the archived large Phase C checkpoint moves exact match from 0.375 to 0.75, tolerance accuracy from 0.375 to 0.75, parseability from 0.641 to 1.0, and average completion length from 196.3 to 25.0 tokens. Table 4 summarizes that before-versus-after comparison.

An important nuance is that the archived dedicated Phase D run matches, rather than exceeds, the archived best Phase C score. In other words, the case-study evidence supports hard-subset specialization as a viable continuation path, but it does not suggest that Stage-3-only specialization clearly dominates the mixed Stage 2/3 continuation.

6.3 Archived checkpoints motivate alias-based selection

The repository's saved runs also show why alias-based checkpoint selection was part of the implementation. In archived Smoke Phase B, the best composite checkpoint reaches exact match 0.5 with perfect structure, while the latest checkpoint regresses to exact match 0.0 despite keeping parseability and tag compliance at 1.0. In the archived same-notebook Phase D branch, the best checkpoint reaches 0.75 exact match but the latest checkpoint ends at 0.5. This is not a theorem about RL in general. It is a concrete property of the observed checkpoint lineage in this repository, and it explains why the author introduced explicit aliases instead of resuming blindly from `latest`.

6.4 THE ARCHIVED CONTROLLER AUDIT IS CONSISTENT WITH THE INTENDED POLICY

The saved controller audit covers 26 evaluated checkpoints across seven phase resets. Its main descriptive result is that the rules fire in the expected order. Parseability, formatting, and finish guards fire only twice each, concentrated in the structurally weak part of the trajectory. Correctness escalation fires 12 times, but only after structure is already stable. Notably, six of those correctness escalations happen on checkpoints where exact match has regressed rather than merely plateaued. In a case-study framing, that behavior is useful because it exposes the controller's bias directly: once structure is considered solved, the system responds to weak accuracy progress by increasing correctness pressure rather than by re-raising structure rewards.

The same combined view in Figure 2 places that audit trail back into the checkpoint lineage. In the single-figure presentation, the controller-side panels remain available without splitting the narrative into separate quality and control figures.

6.5 THE SAME LOW-MEMORY PROFILE UNDERLIES THE ARCHIVED 0.75 RESULT

Table 2 and the saved runtime-tuning tables make a second descriptive point clearer. The archived large continuation does not coincide with a more permissive hardware regime. The successful smoke and large-split runs share the same Kaggle T4 profile: sequence length 1280 instead of 16384, image size 336 instead of 512, LoRA rank 8 instead of 16, and two sampled generations instead of four. The cost is a shorter reasoning budget and lower generation diversity. The benefit is a stable single-GPU operating regime that supports both early curriculum validation and later continuation without changing the basic VRAM regime.

7 DISCUSSION

The main value of this report is descriptive rather than universal. The repository suggests a layered story for one compute-constrained multimodal RL project.

First, a strict response contract makes structural failures measurable. Second, structure-oriented reward components give the optimizer a path to improve those failures before exact match is reliable. Third, a checkpoint-metric controller stops correctness reward from dominating too early. Fourth, alias-based checkpoint selection acknowledges that the saved trajectory is non-monotonic. Taken together, these choices turn the training loop into something that is easier to audit and reason about.

At the same time, the case-study framing imposes discipline on the claims. We do not claim that the archived 0.75 checkpoints establish a new state of the art. We do not claim that the controller is uniquely responsible for every improvement, because the repository does not contain the ablations required for that conclusion. The narrower claim is that the archived artifacts are internally consistent with a structure-first training recipe and that the implementation makes those dynamics unusually legible.

8 LIMITATIONS AND REPRODUCIBILITY

The limitations are substantial and should shape how the report is read.

- The reported numbers are archived checkpoint-side measurements from the original runs, not an external benchmark evaluation.
- The monitored evaluation subset is tiny in the Kaggle profile, so the quantitative results are useful for repository analysis but weak for broad empirical comparison.
- The split protocol is repository-specific and, in smoke runs, not held out from training.
- The codebase studies one base model family and one principal hardware profile.
- The repository does not include the seed sweeps or controlled ablations needed to turn this case study into a strong empirical method paper.
- The multi-choice branch (Phase E) and multilingual robustness stage are scaffolded but not part of the main evidence.

Despite those limitations, the repository is unusually reproducible for a small RL project. It logs controller decisions at every evaluated checkpoint, persists multiple checkpoint-selection aliases, saves sample-level traces, and includes unit tests for the most failure-prone logic. Those properties are exactly why the project works better as a case study than as an over-claimed benchmark paper. The separate supplement expands that reproducibility story with explicit reward equations, evaluation formulas, controller thresholds, and transfer semantics.

9 Conclusion

We presented a code-grounded case study of staged metric-gated GRPO for visual numeric reasoning. The archived run artifacts show a simple but practically useful pattern: structure must become reliable before correctness reward becomes consistently useful. In this repository, a phase-wise curriculum, metric-gated reward controller, and alias-based checkpoint selection are associated with a transition from a malformed, long-output baseline to archived best checkpoints that record exact match 0.75 under the repository's internal evaluation protocol. That is not a broad benchmark claim. It is a reproducible report of how one staged RL system behaved, why the author organized it that way, and which parts of the implementation appear to matter most.

Supplement. The companion supplement contains the full reward and evaluation formulas, controller-rule reference, hardware-profile details, and supporting implementation figures.

References

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, Junyang Lin, et al. Qwen2.5-vl technical report, 2025. URL https://arxiv.org/abs/2502.13923.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.02255.

Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.18252.

Ji Qi, Ming Ding, Weihan Wang, Yushi Bai, Qingsong Lv, Wenyi Hong, Bin Xu, Lei Hou, Juanzi Li, Yuxiao Dong, and Jie Tang. Cogcom: A visual language model with chain-of-manipulations reasoning. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2402.04236.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Yue Wu, Zhiqing Sun, Huizhuo Yuan, Kaixuan Ji, Yiming Yang, and Quanquan Gu. Self-play preference optimization for language model alignment. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2405.00675.

Renrui Zhang, Xinyu Wei, Dongzhi Jiang, Ziyu Guo, Yichi Zhang, Chengzhuo Tong, Jiaming Liu, Aojun Zhou, Shanghang Zhang, Peng Gao, and Hongsheng Li. Mavis: Mathematical visual instruction tuning with an automatic data engine. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2407.08739.